DELHI TECHNOLOGICAL UNIVERSITY

(Formerly Delhi College of Engineering) Shahbad

Daulatpur, Bawana Road, Delhi-110042

DEPARTMENT OF COMPUTER SCIENCE

AND ENGINEERING

Machine Learning Lab File

CO303

Submitted To:

Assistant Prof. Gull Kaur

Department of Computer

Science and Engineering

Submitted By:

Krish Bansal

24/CS/248

INDEX

SNo.	Aim	Date	Page No.	Signature
1	To implement and evaluate the K-Nearest Neighbors (KNN) classification algorithm on the Iris dataset.			
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				

EXPERIMENT – 1

AIM:

To implement and evaluate the **K-Nearest Neighbors (KNN) classification algorithm** on the Iris dataset. The experiment aims to explore the dataset, visualize its features, train KNN classifiers for different values of (k), compare their training and testing accuracies, study the effect of feature scaling, calculate class probabilities, and visualize decision boundaries.

THEORY:

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression. In classification, KNN assigns a class to a new data point based on the classes of its (k) nearest training samples.

The closeness between data points is generally calculated using **Euclidean distance**:

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

The value of (k) determines the number of neighboring samples considered for classification.

- A small value of (k) can result in low bias but high variance and may lead to overfitting.
- A larger value of (k) generally produces smoother decision boundaries but may increase bias.
- Feature scaling is important in KNN because the algorithm depends directly on distances between feature vectors. Features having larger numerical ranges can otherwise dominate the distance calculation.
- predict_proba() can be used to estimate the probability of a test sample belonging to each class based on its neighboring samples.

The **Iris dataset** contains 150 samples belonging to three classes:

1. Iris Setosa
2. Iris Versicolor

3. Iris Virginica

Each sample contains four features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

The dataset is divided into **70% training data and 30% testing data**.

CODE:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.inspection import DecisionBoundaryDisplay

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib as mpl

# Load Iris dataset
dataset = load_iris()

features = dataset.data
target = dataset.target

print(dataset.DESCR)
```

```
# Convert dataset into DataFrame
ds_pandas = pd.DataFrame(
    features,
    columns=dataset.feature_names
)

ds_pandas['target'] = target

print(ds_pandas.head())

# Visualize feature relationships
sns.pairplot(ds_pandas, hue="target")
plt.show()

# Plot class distribution
sns.displot(ds_pandas, x="target")
plt.show()

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    features,
    target,
    random_state=0,
    test_size=0.30
)

# KNN without feature scaling
for k in [1, 3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
```

```
    knn.fit(X_train, y_train)

    train_acc = knn.score(X_train, y_train)
    test_acc = knn.score(X_test, y_test)

    print(
        f"Neighbors: {k} | "
        f"Training Accuracy: {train_acc:.4f} | "
        f"Testing Accuracy: {test_acc:.4f}"
    )

# Class probability
class_probability = knn.predict_proba(X_test)
print(class_probability)

# Feature scaling
scaler = StandardScaler()

scaler.fit(X_train)

X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)

# KNN after scaling
for k in [1, 3, 5, 7]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)

    train_acc = knn.score(X_train_scaled, y_train)
```

```
    test_acc = knn.score(X_test_scaled, y_test)

    print(
        f"Neighbors: {k} | "
        f"Training Accuracy: {train_acc:.4f} | "
        f"Testing Accuracy: {test_acc:.4f}"
    )

# Class probabilities after scaling
class_probability = knn.predict_proba(X_test_scaled)
print(class_probability)

# Decision boundary visualization
X_train_2d = X_train[:, :2]
X_test_2d = X_test[:, :2]

for k in [1, 3, 5, 7]:

    clf = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=k))
    ])

    _, axs = plt.subplots(ncols=2, figsize=(12, 5))

    for ax, weights in zip(
        axs, ("uniform", "distance")
    ):
```

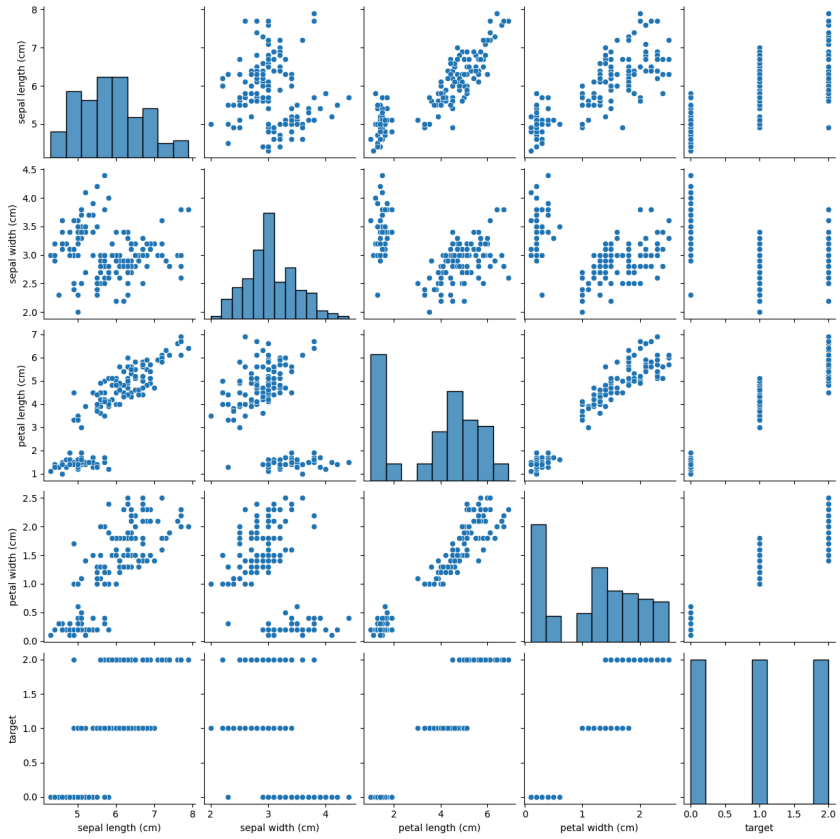
```
clf.set_params(
    knn__weights=weights
).fit(X_train_2d, y_train)

DecisionBoundaryDisplay.from_estimator(
    clf,
    X_test_2d,
    response_method="predict",
    plot_method="pcolormesh",
    xlabel=dataset.feature_names[0],
    ylabel=dataset.feature_names[1],
    shading="auto",
    alpha=0.5,
    ax=ax
)

scatter = ax.scatter(
    X_test_2d[:, 0],
    X_test_2d[:, 1],
    c=y_test,
    edgecolors="k"
)

ax.legend(
    scatter.legend_elements()[0],
    dataset.target_names,
    loc="lower right",
    title="Classes"
)
```

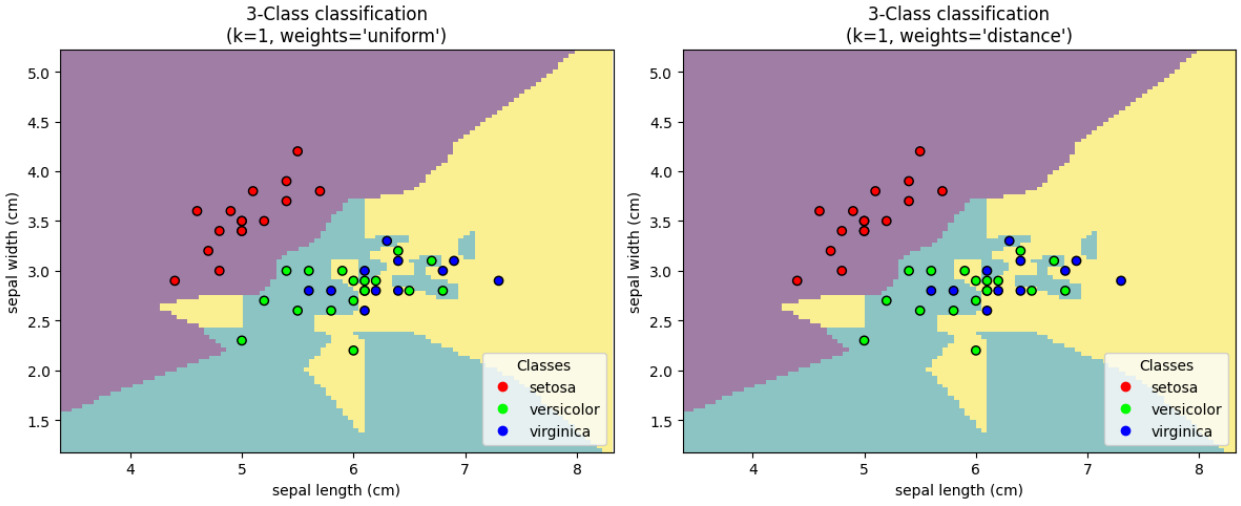
```
        ax.set_title(
            f"3-Class Classification "
            f"(k={k}, weights={weights})"
        )

plt.tight_layout()
plt.show()
```

OUTPUT:

The Iris dataset contains:

- Total samples: **150**
- Number of features: **4**
- Number of classes: **3**
- Training samples: **105**
- Testing samples: **45**


sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
target


3-Class classification
(k=1, weights='uniform')
3-Class classification
(k=1, weights='distance')
sepal length (cm)
sepal width (cm)
Classes
setosa
versicolor
virginica

KNN Without Feature Scaling

K Value	Training Accuracy	Testing Accuracy
1	1.0000	0.9778
3	0.9619	0.9778
5	0.9714	0.9778
7	0.9714	0.9778

All tested values of (k) achieved approximately **97.78% testing accuracy**. The model with (k=1) achieved 100% training accuracy, indicating that a very small value of (k) can closely fit the training data.

KNN After Feature Scaling

K Value	Training Accuracy	Testing Accuracy
1	1.0000	0.9333
3	0.9714	0.9778
5	0.9714	0.9778
7	0.9714	0.9778

After applying StandardScaler, the models with (k=3,5,7) continued to achieve approximately **97.78% testing accuracy**, while (k=1) achieved approximately **93.33% testing accuracy**.

The predict_proba() function generated probabilities representing the proportion of neighboring samples belonging to each Iris class.

The pairplot showed that the different Iris species can be distinguished using their feature values, with petal measurements providing particularly clear separation.

Decision-boundary plots were also generated for (k=1,3,5,7) using both **uniform weighting** and **distance-based weighting**.

LEARNING OUTCOME:

After completing this experiment, I was able to:

1. Load and explore the Iris dataset using Scikit-learn and Pandas.
2. Visualize feature relationships and class distributions using Seaborn.
3. Understand and implement the K-Nearest Neighbors classification algorithm.
4. Perform a train-test split and evaluate a machine learning classifier.
5. Understand the effect of changing the value of (k) on KNN performance.
6. Understand the importance of feature scaling in distance-based algorithms such as KNN.
7. Calculate and interpret class probabilities using predict_proba().
8. Compare uniform and distance-based neighbor weighting.
9. Visualize and interpret KNN decision boundaries.